

1. Setting up `package.json`

- Start with a standard React app:

```
{
  "name": "frontend",
  "version": "0.1.0",
  "private": true,
  "dependencies": { /* all dependencies */ },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  }
}
```

- Ensure `build` script exists as we will deploy the `build/` folder.

2. AWS S3 Bucket Setup

1. Create a bucket (`letsworkout`) in the desired region (e.g., `ap-south-1`).
2. Enable public access if using CloudFront for public hosting.
3. Note the bucket ARN: `arn:aws:s3:::letsworkout`

3. AWS CloudFront Distribution

1. Create a CloudFront distribution with S3 as the origin.
2. Enable `Grant CloudFront access to origin` to allow policy updates automatically.
3. Note the distribution ID, e.g., `E1HLQPG5MI6CKF`.
4. Update S3 bucket policy to allow CloudFront:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {"Service": "cloudfront.amazonaws.com"},
      "Action": "s3:GetObject",

```

```
    "Resource": "arn:aws:s3:::letsworkout/*",
    "Condition": {"StringEquals": {"AWS:SourceArn":
"arn:aws:cloudfront::<account-id>:distribution/E1HLQPG5MI6CKF"}}
  }
]
}
```

4. IAM User Setup

1. Create IAM user `letsworkout-developer` with **Programmatic access**.
2. Attach policies:
3. `AmazonS3FullAccess`
4. `CloudFrontFullAccess`
5. Generate **Access Key ID** and **Secret Access Key**.
6. Store them in GitHub repository secrets:
7. `AWS_ACCESS_KEY_ID`
8. `AWS_SECRET_ACCESS_KEY`

5. Firebase & Google Sign-in Notes

- Ensure your CloudFront domain is authorized in Firebase Authentication → Settings → Authorized Domains.
- For backend running on Firebase, Google sign-in works after adding the CloudFront URL.

6. Initial GitHub Actions Workflow

- `.github/workflows/deploy.yml` basic version:

```
name: Deploy React App to S3 + CloudFront
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: {node-version: '18'}
      - run: npm install
      - run: npm run build
      - uses: aws-actions/configure-aws-credentials@v2
```

```

    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: ap-south-1
    - run: aws s3 sync build/ s3://letsworkout --delete
    - run: aws cloudfront create-invalidation --distribution-id
      E1HLQPG5MI6CKF --paths "/*"

```

- Builds React app, uploads to S3, invalidates CloudFront cache.

7. Handling CI Warnings (crypto issue)

- On GitHub Actions, React treats warnings as errors because `process.env.CI = true`.
- Fix in workflow:

```

- name: Build React app
  run: |
    set -e
    CI=false npm run build

```

- Ensures warnings (like Node crypto polyfill) don't break build, but real errors still fail.

8. Optimized CI/CD Workflow with Caching

- Caches npm dependencies to speed up builds.
- Verifies build folder exists.
- Full workflow:

```

name: Deploy React App to S3 + CloudFront
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: {node-version: '18'}
      - name: Cache npm dependencies
        uses: actions/cache@v3
        with:
          path: ~/.npm

```

```

    key: ${{ runner.os }}-node-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-
  - run: npm install
  - name: Build React app
    run: |
      set -e
      CI=false npm run build
  - name: Verify build folder
    run: |
      if [ ! -d "build" ]; then
        echo "Error: build folder not found!"
        exit 1
      fi
      echo "Build folder exists."
      ls -R build/
  - uses: aws-actions/configure-aws-credentials@v2
    with:
      aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
      aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
      aws-region: ap-south-1
  - run: aws s3 sync build/ s3://letsworkout --delete
  - run: aws cloudfront create-invalidation --distribution-id
    E1HLQPG5MI6CKF --paths "/*"

```

9. Best Practices Learned

1. Use **dedicated IAM user** for CI/CD, not root credentials.
 2. Use **GitHub Secrets** to store AWS keys securely.
 3. Set `CI=false` in React build to prevent warnings breaking CI.
 4. Use **npm caching** to speed up repeated builds.
 5. Always **verify build folder** exists before deploying.
 6. Invalidate CloudFront cache to serve updated content immediately.
 7. Optionally, in future, use **OIDC GitHub Actions integration** for better security.
-

This document now serves as a **reference guide** for creating a CI/CD pipeline for a React app from scratch using AWS S3, CloudFront, IAM, and GitHub Actions, including optimized practices for speed and reliability.